

Mobile Backend Architecture & Performance Testing

(Flash Sale System)

วัตถุประสงค์:

ให้นักศึกษาทำงานเป็นกลุ่ม (กลุ่มละ 3 คน) เพื่อพัฒนาระบบ Backend สำหรับแอปพลิเคชันบนมือถือในสถานการณ์ "Flash Sale" ที่มีผู้ใช้งานจำนวนมากเข้ามาดูสินค้าและกดสั่งซื้อพร้อมกัน ระบบจะต้องออกแบบมาให้รองรับปริมาณ Request มหาศาลได้อย่างมีประสิทธิภาพ (High Throughput & Low Latency) ป้องกันปัญหาการสั่งซื้อสินค้าเกินจำนวนที่มีอยู่จริง (Race Condition) รวมไปถึงการจัดการระบบ Authentication และ Caching อย่างถูกต้อง

1. ข้อกำหนดทางสถาปัตยกรรม (Architecture Requirements)

ระบบของแต่ละกลุ่มจะต้องประกอบด้วยองค์ประกอบดังต่อไปนี้ และต้อง Deploy ร่วมกันผ่าน docker-compose.yml:

- **Load Balancer:** ใช้ Nginx กระจายโหลด (Load Balancing) แบบ Round Robin หรือ Least Connections ไปยัง Backend อย่างน้อย 3 Instances
- **Backend Application:** พัฒนาด้วย NestJS โดยใช้โครงสร้างแบบ Modular
- **Database:** PostgreSQL (ใช้ TypeORM) พร้อมกำหนด Connection Pooling ให้เหมาะสม
- **Caching & Invalidation:** ใช้ Redis สำหรับทำ Caching ข้อมูลสินค้า
- **Message Queue:** ใช้ BullMQ (บน Redis) สำหรับประมวลผลการสั่งซื้อแบบ Asynchronous (เช่น การตัดสต็อก)
- **Stateless Authentication:** ใช้ JWT (JSON Web Token) สำหรับยืนยันตัวตน ห้ามใช้ In-memory sessions เด็ดขาด เพื่อให้รองรับ Horizontal Scaling ได้
- **Observability Dashboard:** มี Dashboard สำหรับ Monitor ระบบ (เช่น หน้า Bull-Board สำหรับดูคิว)

2. มาตรฐาน API Endpoints (Standardized API Specs)

สำคัญ: ทุกกลุ่มจะต้อง Implement API ตามสเปกด้านล่างนี้อย่างเคร่งครัด เพื่อให้สามารถใช้ Load Test Script ร่วมกันระหว่างกลุ่มได้

2.1. API สำหรับขอ Token (Authentication) Authenticate จะไม่ได้วัด performance

จำลองการ Login เพื่อรับ JWT นำไปใช้ใน API สั่งซื้อสินค้า

- **Endpoint:** POST /api/v1/auth/token
- **Request Body:**
JSON
{ "userId": "user-999" }
- **Response (200 OK):**
JSON
{
 "status": "success",

```
"accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6Ikp.."
```

2.2. API สำหรับดึงข้อมูลสินค้า (Read-Heavy)

API นี้จะถูกยิง Load Test ด้วยปริมาณมหาศาล **ต้องมีการใช้ Redis Caching** (เช่น Cache-Aside) เพื่อให้ Response เร็วที่สุด และต้องใช้เทคนิค Pagination สำหรับผลลัพธ์ที่มีจำนวนมาก

- **Endpoint:** GET /api/v1/products?page=1&limit=10
- **เงื่อนไขสำคัญ (Cache Invalidation):** API นี้ต้องคืนค่า remainingStock ที่ถูกต้องเสมอ ดังนั้นเมื่อมีการดัดสต็อกสำเร็จในฝั่ง Worker (Queue) ระบบจะต้องมีกลยุทธิ์การทำ Cache Invalidation เพื่อลบหรืออัปเดต Cache ของหน้านั้นๆ ทันที

- **Response (200 OK):**

JSON

```
{
  "status": "success",
  "data": [
    {
      "productId": "p-1001",
      "name": "Limited Edition Sneaker",
      "price": 2990,
      "availableStock": 50,
      "remainingStock": 30,
      "isFlashSaleActive": true
    }
  ],
  "meta": {
    "total": 20,
    "page": 1,
    "limit": 10,
    "totalPages": 2
  }
}
```

2.3. API สำหรับส่งซื้อสินค้า (Write-Heavy / Concurrent)

API นี้จำลองการกดจองสินค้า ต้องมีการตรวจสอบ JWT และต้องส่งงานเข้า Message Queue (BullMQ) ทั้งนี้จากนั้นตอบกลับ Client อย่างรวดเร็ว ห้ามทำ Synchronous Database Update ที่ใช้เวลานานใน Controller

- **Endpoint:** POST /api/v1/orders
- **Headers:** Authorization: Bearer <JWT_ACCESS_TOKEN>
- **Request Body:**

JSON

```
{
  "productId": "p-1001"
}
```

(หมายเหตุ: ไม่ต้องส่ง quantity เพราะถูกบังคับให้ซื้อได้แค่ 1 ชิ้น)

- **Response (202 Accepted):** (ตอบกลับเมื่อรับงานเข้าคิวสำเร็จ)

JSON

```
{  
  "status": "processing",  
  "orderId": "job-12345",  
  "message": "Your order is in the queue."  
}
```

- **เงื่อนไขการทำงาน (Business Rules & Worker Constraints):**

1. **Limit 1 per User:** ผู้ใช้งาน 1 คน (อ้างอิงจาก userId ใน JWT) สามารถจองสินค้าแต่ละรหัส (ProductID) ได้ สูงสุดเพียง 1 ชิ้นเท่านั้น
2. **Concurrency Handling (API Level):** นศ. ต้องหาวิธีป้องกันการกรณีที่ User คนเดิมส่ง Request รัวๆ เข้ามาพร้อมกัน (เช่น ใช้ Atomic Operations ของ Redis อย่าง SETNX หรือ SADD เพื่อล็อกสิทธิ์ชั่วคราวก่อนดันเข้าคิว) **บังคับใช้ Atomic Operations**
3. **Concurrency Handling (Worker/Database Level):** Worker ที่รับงานจาก Queue จะต้องลดจำนวนสต็อกใน Database โดยต้องจัดการ Race Condition (เช่น การใช้ Pessimistic/Optimistic Locking) ไม่ให้ **remainingStock** ติดลบ และอาจต้องพิจารณาทำ Unique Constraint ใน Database ระหว่าง user_id และ product_id เพื่อความปลอดภัยขั้นสูงสุด **Locking**
4. **Cache Invalidation:** เมื่ออัปเดต Database สำเร็จ ต้องสั่ง Invalidate Redis Cache ของ Product เพื่อให้ API GET แสดงสต็อกล่าสุดที่ถูกต้อง

3. การทดสอบ Load Test และ Observability

แต่ละกลุ่มจะต้องเขียน Script สำหรับทำ Load Test (แนะนำเครื่องมือ k6) และยิงทดสอบระบบของกลุ่มตนเอง รวมถึงระบบของกลุ่มเพื่อน

ข้อกำหนดในการทำ Load Test:

1. **Preparation Phase:** Script ต้องวนลูปขอ JWT จาก /api/v1/auth/token สำหรับ **ผู้ใช้ที่ไม่ซ้ำกัน 500 คน** (เช่น user-1 ถึง user-500) เพื่อเตรียมไว้ใช้ในขั้นตอนต่อไป
2. **Read Load:** ยิง HTTP GET ไปที่ /api/v1/products?page=1&limit=10 จำนวน 1,000 Concurrent users **ลองเปลี่ยน Page ลองเปลี่ยน limit บ้างก็ดี**
3. **Write Load:** ยิง HTTP POST ไปที่ /api/v1/orders จำนวน 500 Concurrent requests พร้อมแนบ JWT (โดยใช้ JWT ของ User ที่ไม่ซ้ำกัน) เพื่อจำลองคน 500 คนแย่งกันกดซื้อสินค้า p-1001 (ซึ่งมีสต็อกจำกัดเพียง 50 ชิ้น) และอาจมีการจำลองให้ User บางคนยิง Request เบิ้ลมา 2-3 ครั้งพร้อมๆ กัน เพื่อทดสอบระบบป้องกันสิทธิ์ซ้ำซ้อน

สิ่งที่ต้องแสดงใน Dashboard หรือ Report ของกลุ่ม:

1. **Cache Performance:** แสดง Cache Hit / Cache Miss Ratio
2. **Queue Monitoring:** แสดงสถานะของ Worker, จำนวน Jobs In Queue, Completed Jobs, และ **Failed Jobs** (เช่น กรณีของหมดแล้ว หรือ User คนเดิมพยายามซื้อซ้ำ)
3. **Throughput & Latency:** สรุปผล Requests per second (Req/s), p95 Latency และ Error Rate จากเครื่องมือ Load Test

4. **Data Integrity Proof:** แคปเจอร์หน้าจอ Database เพื่อพิสูจน์ว่า:
- remainingStock ของสินค้ามีค่าเป็น 0 พอดี ไม่มีการติดลบ
 - ตารางเก็บประวัติการสั่งซื้อ (Orders) มี Record ของผู้ใช้ 50 คนที่ไม่ซ้ำหน้ากันเลย และไม่มีใครได้สินค้าเกิน 1 ชิ้น

4. สิ่งที่ต้องส่ง (Deliverables)

1. **Source Code:** ลิงก์ GitHub Repository ที่มี docker-compose.yml สำหรับการรันแบบ 1-click start
2. **Load Test Script:** ไฟล์ Script (เช่น loadtest.js สำหรับ k6) ที่ใช้ทดสอบ (วางไว้ Github Repository เดียวกัน)
3. **Report (PDF):**
 - Diagram สถาปัตยกรรมของระบบ
 - คำอธิบายกลยุทธ์การทำ Cache Invalidation และการป้องกันผู้ใช้ส่งของซ้ำซ้อน
 - ผลลัพธ์จาก Load Test Dashboard (แคปเจอร์หน้าจอพร้อมอธิบาย)
 - ตารางเปรียบเทียบผลลัพธ์: การยิง Load test ใส่ API กลุ่มตัวเอง เทียบกับผลการยิงใส่ API ของกลุ่มเพื่อน (วิเคราะห์สาเหตุหากเกิดคอขวด)
 - ระบุสมาชิกในกลุ่ม และการแบ่งหน้าที่การทำงานของสมาชิกแต่ละคน